

哈尔滨工业大学（深圳）

软件构造 实验指导书

实验六 观察者和模板模式

2026 春

目录

1. 实验目的.....	3
2. 实验环境.....	3
3. 实验内容.....	3
4. 实验步骤.....	3
4.1 观察者模式应用.....	3
4.1.1 炸弹、冰冻道具场景分析.....	3
4.1.2 代码实现.....	5
4.2 模板模式应用.....	8
4.2.1 难度选择场景分析.....	8
4.2.2 代码实现.....	9
5. 本次迭代开发目标.....	13
6. 期末提交要求.....	14

1. 实验目的

1. 深入理解观察者与模板模式的设计意图及结构
2. 结合案例，规范绘制两种模式的 UML 类图
3. 掌握重构方法，通过代码实现并应用两种设计模式

2. 实验环境

1. IntelliJ IDEA 2025.3
2. OpenJDK 25

3. 实验内容

1. 深入学习观察者与模板模式的设计原理，结合实例完成两种设计模式的 UML 结构图绘制
2. 对系统内的道具生效逻辑与难度控制逻辑进行代码重构：
 - ✓ 采用观察者模式实现炸弹、冰冻道具的功能逻辑
 - ✓ 采用模板模式实现简单、普通和困难三种游戏难度
3. 完成本次迭代开发清单中的全部任务

4. 实验步骤

4.1 观察者模式应用

4.1.1 炸弹、冰冻道具场景分析

在飞机大战游戏中包含 2 种特殊道具：炸弹道具、冰冻道具。两种道具均在敌机被击毁时按预设概率随机掉落生成，英雄机碰撞道具后自动触发生效，炸弹、冰冻道具对敌机子弹及不同类型敌机的效果存在显著差异。具体生效效果如下表所示：

	炸弹道具	冰冻道具
普通敌机	坠毁	永久静止
精英敌机	坠毁	静止 4s 后恢复
精锐敌机	坠毁	静止 3s 后恢复
王牌敌机	掉血	减速 5s 后恢复
Boss 敌机	不受影响	不受影响
敌机子弹	消失	静止 5s 后恢复

注：炸弹道具生效后，英雄机可获得所有坠毁敌机分数，坠毁敌机不掉落道具。

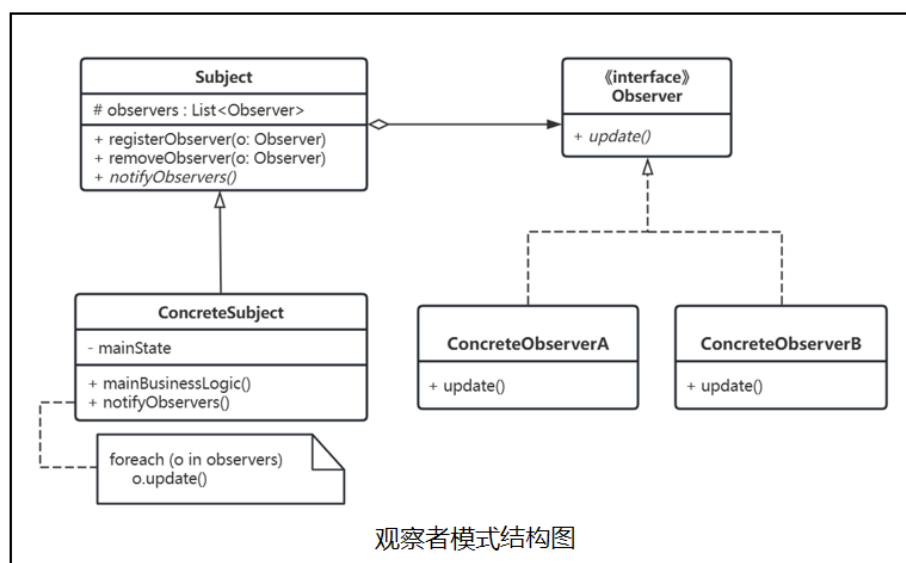
设计任务：

鉴于两种道具生效时需要通知并影响多个不同类型的敌机对象，且对不同敌机的影响逻辑各不相同，为了实现道具与敌机之间的松耦合，避免道具类直接依赖大量具体的敌机类，请采用观察者模式进行道具生效逻辑的设计与代码重构。

UML 建模：绘制该模式的 UML 类图，图中需明确标注设计模式名称，并自定义合理的类名、属性名及方法名，以完整展示观察者模式的实现结构。

观察者模式

观察者模式（Observer Pattern）也是一种行为设计模式，允许你定义一种订阅机制，可在对象事件发生时通知多个“观察”该对象的其他对象。



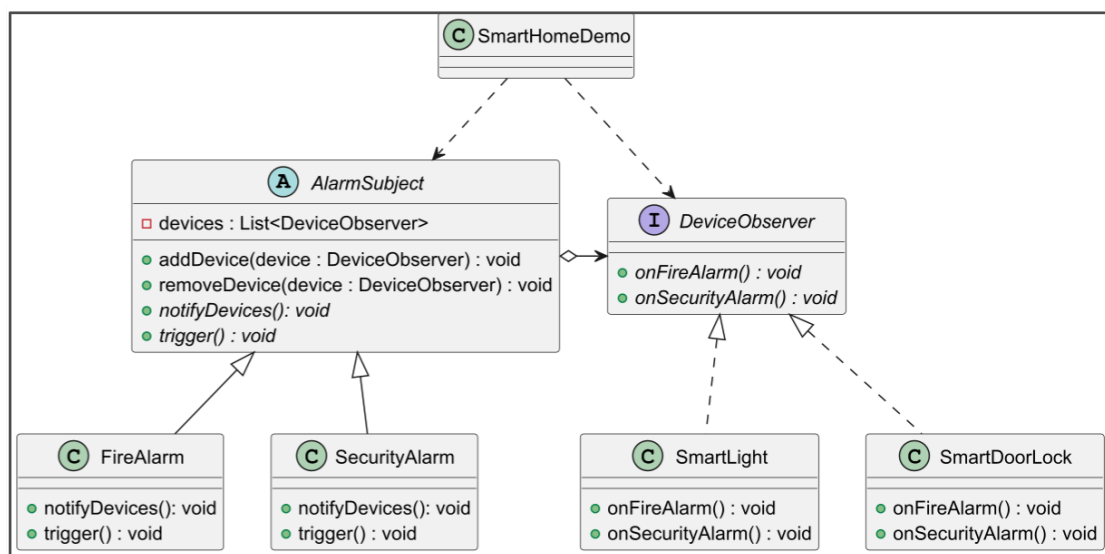
请参考以上 UML 结构图，绘制飞机大战中的观察者模式。

4.1.2 代码实现

观察者模式的代码示例（智能家居警告系统）：

在智能家居警告系统中，当“火灾报警器”触发时，智能灯会打开应急照明，智能门锁会自动解锁，方便人员逃生；当“安防报警器”触发时，智能灯会开启闪烁警示，智能门锁会自动上锁，防止外部入侵。

我们将创建 `AlarmSubject` 抽象类作为抽象观察目标，负责维护智能设备观察者列表，并提供注册和删除观察者的方法。创建 `FireAlarm` 类和 `SecurityAlarm` 类继承自 `AlarmSubject`，作为具体观察目标，分别负责在火灾报警和安防报警触发时通知所有智能设备。创建 `DeviceObserver` 接口充当抽象观察者角色，以及实现了该接口的 `SmartLight` 类和 `SmartDoorLock` 类作为具体观察者。`SmartHomeDemo` 使用这些类和对象来演示观察者模式。



步骤 1: 创建 `DeviceObserver` 接口，充当观察者角色。

`DeviceObserver.java`

```
public interface DeviceObserver {
    // 火灾告警后的反应
    void onFireAlarm();

    // 安防告警后的反应
    void onSecurityAlarm();
}
```

步骤 2: 创建 SmartLight 和 SmartDoorLock 接口实现类，充当具体观察者角色。

SmartLight.java

```
public class SmartLight implements DeviceObserver {

    @Override
    public void onFireAlarm() {
        System.out.println("智能灯: 打开应急照明。");
    }

    @Override
    public void onSecurityAlarm() {
        System.out.println("智能灯: 开启闪烁警示。");
    }
}
```

SmartDoorLock.java

```
public class SmartDoorLock implements DeviceObserver {

    @Override
    public void onFireAlarm() {
        System.out.println("智能门锁: 自动解锁, 方便人员逃生。");
    }

    @Override
    public void onSecurityAlarm() {
        System.out.println("智能门锁: 自动上锁, 防止入侵。");
    }
}
```

步骤 3: 创建 AlarmSubject 类，充当抽象观察目标角色。

AlarmSubject.java

```
import java.util.ArrayList;
import java.util.List;

public abstract class AlarmSubject {
    protected List<DeviceObserver> devices = new ArrayList<>();

    public void addDevice(DeviceObserver device) {
        devices.add(device);
    }

    public void removeDevice(DeviceObserver device) {
        devices.remove(device);
    }
}
```

```

    public abstract void notifyDevices();

    public abstract void trigger();
}

```

步骤 4: 创建 FireAlarm 和 SecurityAlarm 类，充当具体观察目标角色。

FireAlarm.java

```

public class FireAlarm extends AlarmSubject {

    @Override
    public void notifyDevices() {
        for (DeviceObserver device : devices) {
            device.onFireAlarm();
        }
    }

    @Override
    public void trigger() {
        System.out.println("==== 火灾报警器触发! ====");
        notifyDevices();
    }
}

```

SecurityAlarm.java

```

public class SecurityAlarm extends AlarmSubject {

    @Override
    public void notifyDevices() {
        for (DeviceObserver device : devices) {
            device.onSecurityAlarm();
        }
    }

    @Override
    public void trigger() {
        System.out.println("==== 安防报警器触发! ====");
        notifyDevices();
    }
}

```

步骤 5: 使用 SmartHomeDemo 来演示观察者模式的使用法。

SmartHomeDemo.java

```

public class SmartHomeDemo {
    public static void main(String[] args) {
        // 1. 初始化两个观察目标
        AlarmSubject fireAlarm = new FireAlarm();
        AlarmSubject securityAlarm = new SecurityAlarm();

        // 2. 初始化三个观察者
    }
}

```

```

        DeviceObserver light = new SmartLight();
        DeviceObserver doorLock = new SmartDoorLock();

        // 3. 将观察者注册到观察目标中
        fireAlarm.addDevice(light);
        fireAlarm.addDevice(doorLock);

        securityAlarm.addDevice(light);
        securityAlarm.addDevice(doorLock);

        // 4. 触发告警
        fireAlarm.trigger();
        securityAlarm.trigger();
    }
}

```

步骤 6: 执行程序，输出结果：

```

==== 火灾报警器触发! ====
智能灯：打开应急照明。
智能门锁：自动解锁，方便人员逃生。
==== 安防报警器触发! ====
智能灯：开启闪烁警示。
智能门锁：自动上锁，防止入侵。

```

4.2 模板模式应用

4.2.1 难度选择场景分析

在飞机大战游戏中，用户进入游戏界面后，可选择“简单”“普通”“困难”三种游戏难度。选择完成后，系统将加载该难度对应的专属地图，并进行相应的初始参数设定。三种难度的游戏行为存在明显差异，具体影响因素如下：

(1) 初始设定影响因素

- 屏幕中出现的敌机最大数量
- 不同类型敌机的出现概率
- 英雄机的射击周期
- 敌机的射击周期
- 敌机产生周期
- Boss 敌机产生的阈值

(2) 随游戏时间逐步提升的影响因素

- 简单难度：难度不随游戏时间变化，保持初始设定
- 普通难度：敌机产生周期、敌机速度和敌机血量
- 困难难度：敌机产生周期、敌机速度、敌机血量、英雄机射击周期、敌机射击周期

(3) Boss 敌机相关设定

- 简单难度：无法生成 Boss 敌机
- 普通难度：每次召唤 Boss 机时，不改变 Boss 机的血量
- 困难难度：每次召唤 Boss 机时，会提升 Boss 机的血量

这些不同难度的游戏流程具有相同的主循环结构（如初始化、敌机生成、碰撞检测、渲染更新等），但在具体实现细节上存在差异。

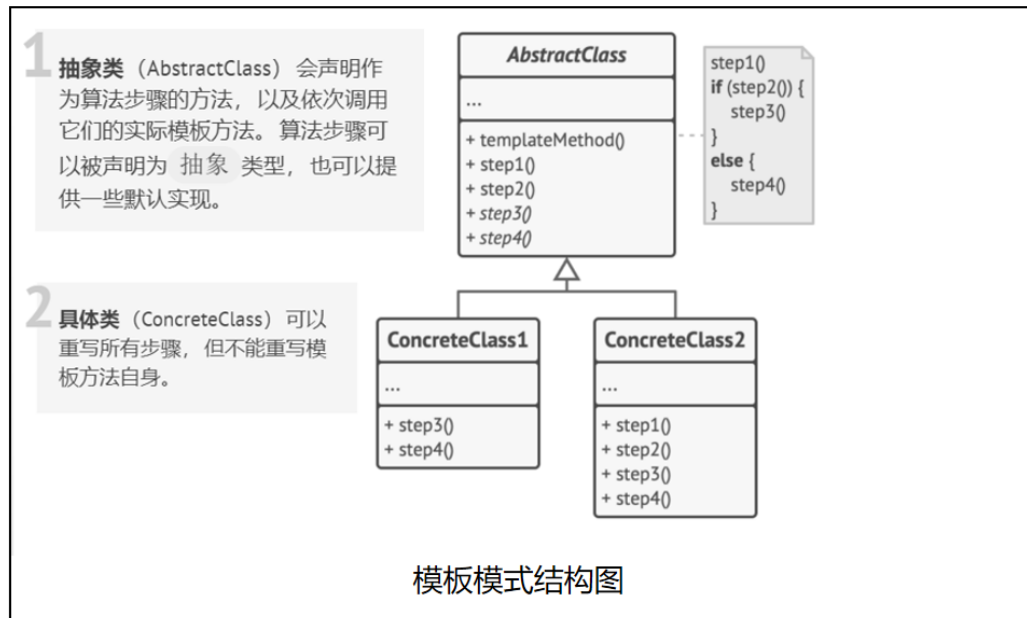
设计任务：

鉴于不同难度下游戏流程的共性与差异，为实现代码复用并保证结构清晰，请采用模板方法模式进行游戏难度类的设计与重构。将游戏主流程定义为算法骨架，将可变步骤交由子类实现。

UML 建模：绘制该模式的 UML 类图，图中需明确标注设计模式名称，并自定义合理的类名、属性名及方法名，以完整展示模板模式的实现结构。

模板模式

模板模式（Template Pattern）是一种行为设计模式，它在超类中定义了一个算法的框架，允许子类在不修改结构的情况下重写算法的特定步骤。



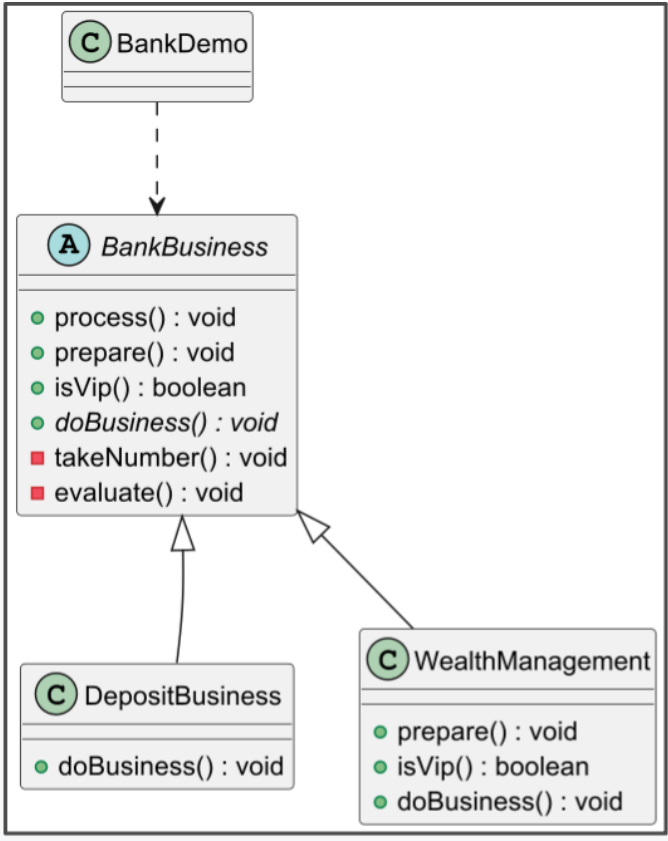
请参考以上 UML 结构图，绘制飞机大战中的模板模式。

4.2.2 代码实现

模板模式代码示例（银行业务办理）：

在银行办理业务时，有一套标准的流程：取号、排队、窗口办理、服务评价。其中“取号”和“服务评价”对所有客户都是固定的。但“排队”和“窗口办理”的内容取决于具体的业务类型（如存款或理财）；同时，系统提供了钩子方法来改变流程：例如，“准备工作”步骤默认不执行，但复杂业务可以重写它；“VIP 身份识别”则是一个逻辑钩子，决定客户是进入普通排队序列还是直接通过 VIP 通道办理。

我们将创建 `BankBusiness` 抽象类作为抽象类角色，负责定义业务办理的模板方法 `process()` 以确定算法骨架，并提供钩子方法 `prepare()` 和逻辑钩子 `isVip()` 供子类挂载或控制流程。创建 `DepositBusiness` 类和 `WealthManagement` 类继承自 `BankBusiness`，作为具体类角色，通过实现抽象方法 `doBusiness()` 来处理具体的业务内容，并根据需要重写钩子方法以实现差异化的办理流程。创建 `BankDemo` 类作为演示类，通过创建不同的业务对象并调用其模板方法，来演示在相同算法结构下，不同业务如何利用抽象步骤与钩子机制实现灵活的逻辑定制。



步骤 1: 创建 `BankBusiness` 抽象类，负责定义业务办理的模板方法，该方法被设置为 `final`。

`BankBusiness.java`

```
public abstract class BankBusiness {

    // 模板方法：定义了办理业务的固定骨架
    public final void process() {
```

```

        prepare();    // 钩子方法 1

        takeNumber(); // 固定步骤

        if (isVip()) {
            System.out.println("VIP 通道：无需等待，直接前往窗口。");
        } else {
            System.out.println("普通通道：请在休息区等候叫号...");
        }

        doBusiness(); // 抽象方法：具体业务内容

        evaluate(); // 固定步骤
    }

    // 钩子方法 1：默认不需要准备材料，子类可挂载
    public void prepare() {}

    // 钩子方法 2：默认不是 VIP，子类可重写返回逻辑
    public boolean isVip() {return false;}

    // 固定步骤：取号
    private void takeNumber() {System.out.println("第 1 步：取号排队。");}

    // 抽象方法：子类必须实现具体的业务逻辑
    public abstract void doBusiness();

    // 固定步骤：评价
    private void evaluate() {System.out.println("最后一步：反馈评价。");}
}

```

步骤 2：创建扩展了上述类的实体类，它们重写了抽象类的某些方法。

普通存款业务 (DepositBusiness)：流程简单，不需要准备，也不是 VIP。

DepositBusiness.java

```

public class DepositBusiness extends BankBusiness {

    @Override
    public void doBusiness() {
        System.out.println("核心业务：办理现金存款 5000 元。");
    }

    // 采用父类默认的钩子：不准备材料，非 VIP
}

```

大额理财业务 (WealthManagement)：重写了钩子方法，改变了算法流程。

WealthManagement.java

```

public class WealthManagement extends BankBusiness {

```

```

@Override
public void prepare() {
    System.out.println("前置准备：填写《风险测评问卷》和《大额交易申报单》。");
}

@Override
public boolean isVip() {
    // 重写钩子，使其走 VIP 逻辑分支
    return true;
}

@Override
public void doBusiness() {
    System.out.println("核心业务：购买 100 万元大额理财产品。");
}
}

```

步骤 3: 使用 BankDemo 的模板方法 process() 来演示模板模式的使用法。

BankDemo.java

```

public class BankDemo {
    public static void main(String[] args) {
        System.out.println("=== 场景一：用户 1 办存款 ===");
        BankBusiness user1 = new DepositBusiness();
        user1.process();

        System.out.println("\n=== 场景二：用户 2 办理财 ===");
        BankBusiness user2 = new WealthManagement();
        user2.process();
    }
}

```

步骤 4: 执行程序，输出结果：

```

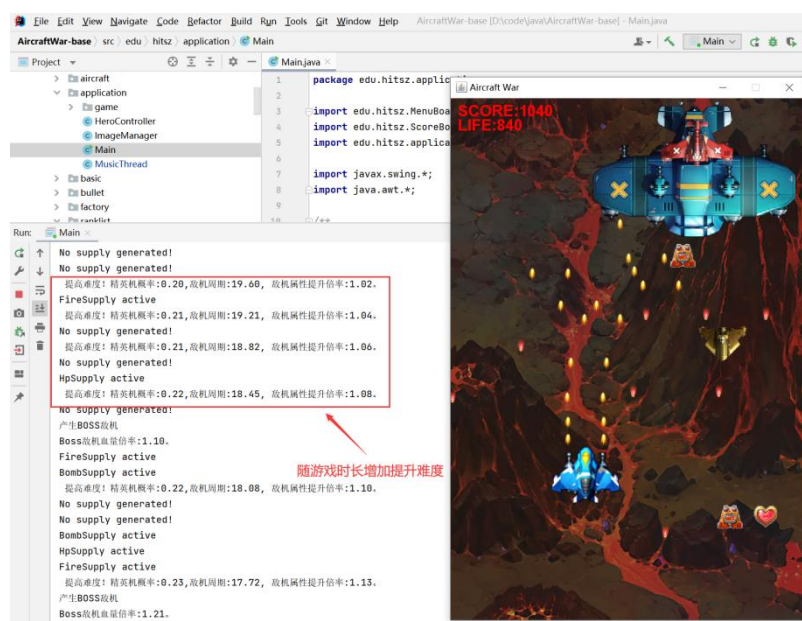
=== 场景一：用户 1 办存款 ===
第 1 步：取号排队。
普通通道：请在休息区等候叫号...
核心业务：办理现金存款 5000 元。
最后一步：反馈评价。

=== 场景二：用户 2 办理财 ===
前置准备：填写《风险测评问卷》和《大额交易申报单》。
第 1 步：取号排队。
VIP 通道：无需等待，直接前往窗口。
核心业务：购买 100 万元大额理财产品。
最后一步：反馈评价。

```

5. 本次迭代开发目标

- 1) 本次迭代任务清单（Checklist）：
- 2) （1）**UML 类图绘制**：按照规范绘制观察者模式与模板模式的结构类图
（2）**设计模式应用**：完成代码重构，使用观察者模式实现炸弹、冰冻道具，使用模板模式实现三种游戏难度
- 注：实现模板模式时，要求将 **Game** 类的 **action** 方法重构为模板方法
- 3) （3）**道具效果**：确保英雄机碰撞炸弹、冰冻道具后，可自动触发对应生效效果；不同道具对不同类型敌机的作用效果存在显著差异，具体效果详见 4.1.1 炸弹、冰冻道具场景分析
- 4) （4）**难度选择功能**：设计并实现“简单”、“普通”和“困难”三种游戏难度的选择功能，三种难度的游戏需符合场景设定，存在明显差异，具体要求详见 4.2.1 难度选择场景分析。
- 5) （5）**Boss 敌机控制**：简单难度无法生成 Boss 敌机；普通难度每次召唤 Boss 机不改变其血量；困难难度每次召唤 Boss 机提升其血量。
- 6) （6）**难度递进功能**：实现“普通”和“困难”两种模式的难度递进逻辑，确保游戏难度随游戏时间逐步递增，具体递增规则详见 4.2.1 难度选择场景分析。需在控制台输出相应信息，参考输出如下：



6. 期末提交要求

✧ 提交网址

提交至作业提交平台（截止日期参考平台发布），请使用统一身份认证登录，推荐使用 Chrome 浏览器。

作业平台入口：<https://grader.hitsz.edu.cn/login?next=/courses>

✧ 提交内容

- 1) 整个项目压缩包：将整个项目压缩为 zip 格式提交，需包含全部代码、测试代码、UML 类图等相关文件
- 2) 实验报告：按照给定的实验报告模板完成撰写
- 3) 录制视频：仅完成创新功能的同学需要录制，视频时长小于 2 分钟，需清晰展示创新功能。

注：创新功能可加分，请在实验报告中说明并提交视频；总分不超过 100 分。